

Conf-ReAct: User-Controlled Tradeoffs in LLM Agent Reasoning via Conformal Prediction

Anonymous submission

Abstract

Large language models (LLMs) are increasingly used as decision-making and planning agents in interactive text-based reasoning and game-playing environments, where they must repeatedly reason about an evolving state and select actions under uncertainty. Existing approaches largely rely on the LLM’s own confidence signals or iterative self-critique (e.g., ReAct, Reflexion) to guide this process, offering little formal guarantee that the agent’s beliefs about the environment or its decisions are trustworthy. We introduce **Conf-ReAct**, a conformal-prediction-based framework that equips LLM-driven agents with a statistically calibrated confidence set over plausible next actions at each state. We introduce a learned nonconformity score function that maps each (state, action) pair to a calibrated measure of fit, trained via binary cross-entropy against back-labeled trajectory outcomes. Using this score, we construct state-conditional conformal prediction sets over the LLM’s candidate actions that adapt the pruning threshold to the local difficulty of the current state, retaining only a calibrated subset of actions for exploration via a lightweight breadth-first search. We evaluate **Conf-ReAct** on three interactive reasoning environments — WebShop, Game of 24, and ScienceWorld — using three open-source language models ranging from 3B to 12B parameters, comparing against ReAct, Reflexion, ReFlAct, and Rollback. At a 90% target coverage level, **Conf-ReAct** improves task-success rate over the strongest baseline by up to 25% on WebShop, 35% on Game of 24, and 7% on ScienceWorld, while exposing a predictable, single-parameter (α) trade-off between success rate and inference cost.

1 Introduction

The recent emergence of agentic AI systems relies heavily on the reasoning capabilities of large language models (LLMs), which increasingly solve complex, long-horizon tasks by iteratively reasoning, planning, and interacting with an environment. Since decisions made at each state can substantially alter an agent’s subsequent trajectory, it must select a set of viable actions that keeps it on a plausibly successful path without deviating far from it. Recent prompting methods for such decision-making fall into two

broad categories: single-trajectory exploration, exemplified by Chain-of-Thought (Wei et al. 2022), ReAct (Yao et al. 2023b), and ReFlAct (Kim et al. 2025), and multi-trajectory exploration, represented by Reflexion (Shinn et al. 2023) and Rollback (Li and Others 2025); both have shown that coupling language-based reasoning with environment interaction substantially improves performance on sequential decision-making tasks, including embodied environments (e.g., ScienceWorld (Wang et al. 2022)) and web navigation (e.g., WebShop (Yao et al. 2022)). A fundamental challenge shared by these methods is selecting, at every state, an action that is not only correct but also keeps the resulting trajectory short. Exhaustively exploring all candidate actions causes the search space to grow exponentially, making planning computationally expensive and often infeasible over long horizons, while restricting the agent to a single highest-scoring action risks prematurely discarding promising alternatives and causing failures. This creates an inherent trade-off between task-completion rate and the average number of LLM calls made.

Addressing this trade-off requires a principled way to decide, at each state, *how many* candidate actions to retain and *which* ones. Existing methods typically rely on heuristic confidence estimates or additional reasoning to make this decision, offering limited statistical justification for the resulting exploration strategy. We argue instead that action selection should be viewed as an *uncertainty estimation* problem. Rather than asking an agent to identify a set of best actions, we seek a calibrated set of plausible actions that is small enough to enable efficient exploration.

In this work, we propose **Conf-ReAct** (Figure 1), a conformal-prediction-based framework for uncertainty-aware action selection in LLM agents. At each planning step, the LLM first proposes a shortlist of candidate actions. A learned nonconformity scoring function then estimates the likelihood that each candidate belongs to a successful trajectory. Rather than executing all proposed actions or committing to a single prediction, **Conf-ReAct** applies conformal calibration to retain only a statistically calibrated subset of actions, which are subsequently explored using a lightweight breadth-first search strategy. We further introduce a *state-conditional* calibration procedure that adapts the conformal threshold to the local difficulty of the current decision state, allowing uncertain states to retain larger

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

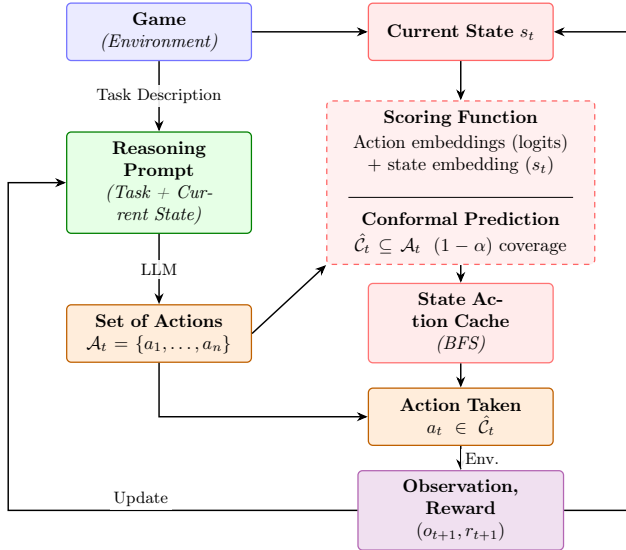


Figure 1: Overview of the proposed conformal action-selection pipeline.

action sets while aggressively pruning easier ones. Unlike existing agentic frameworks, which expose little direct control over the trade-off between task success and inference cost, **Conf-ReAct** allows practitioners to explicitly choose the desired balance, through α - the target miscoverage level. Smaller values of α retain larger action sets, increasing the probability of preserving the optimal trajectory at the cost of additional exploration, whereas larger values encourage more aggressive pruning and lower computational cost.

We evaluate **Conf-ReAct** on three interactive reasoning environments – WebShop, Game of 24, and ScienceWorld – using three recent open-source language models ranging from 3B to 12B parameters. Across all environments, **Conf-ReAct** consistently improves task completion rates over ReAct, Reflexion, ReflAct, and Rollback while exposing a predictable trade-off between success rate and inference cost through the single parameter α . Empirical analysis further shows that the learned conformal action sets achieve substantially higher coverage of the optimal action while remaining significantly smaller than naive exploration strategies, validating the effectiveness of calibrated uncertainty for sequential decision making.

2 Related Work

LLM-Agentic Frameworks: ReAct (Yao et al. 2023b) introduced a framework in which large language models interleave reasoning traces with environment actions, enabling multi-step decision-making in interactive settings. While this paradigm has become foundational for agentic LLM systems, actions are sampled without calibrated uncertainty estimates, and early errors can propagate through trajectories. Our work builds on ReAct but introduces statistically grounded uncertainty quantification for action selection, enabling principled and cost-aware exploration within this framework. Several works extend ReAct-style agents with post-hoc correction

mechanisms rather than calibrated action selection. Reflexion (Shinn et al. 2023) converts sparse environment feedback into a natural-language self-critique after a failed episode, appending it to context for the next trial, but this requires re-reading the entire prior trajectory and multiplies LLM calls per episode. ReflAct (Kim et al. 2025) instead grounds each reasoning step in an explicit comparison between the agent’s current state and its goal, improving plan faithfulness to the true environment state, but still commits to a single proposed action at each step without any calibrated confidence. Rollback (Li and Others 2025) pairs a generator with an assistant model that evaluates each action and triggers a rollback to a previous state upon detecting an error, correcting mistakes only after they have already been committed and evaluated. In contrast, **Conf-ReAct** prevents low-confidence actions from being committed in the first place, using a lightweight learned score function to prune the candidate set to a statistically calibrated subset *before* execution, avoiding the added latency of reflection, the state-drift ReflAct targets, and the reactive correction Rollback relies on. Other lines of work focus on structured plan generation and hierarchical control for agents that must compose multiple tool calls or primitive actions to accomplish long-horizon tasks (Erdogan et al. 2025; Zhao et al. 2024). These agentic architectures emphasize correctness of action selection and the faithful execution of environment APIs, but typically do not explicitly address calibration or guaranteed reliability of the agent’s action set under sampling variability – a gap we address with conformal-style guarantees and self-consistency aggregation in interactive environments.

LLM Conformal Prediction: Uncertainty estimation has been widely studied in supervised learning, with conformal prediction providing distribution-free, finite-sample coverage guarantees. Recent work has applied conformal methods to language models for classification and factuality verification, providing calibrated prediction sets for textual outputs. However, the integration of conformal guarantees into sequential decision-making for LLM agents remains largely unexplored. Gibbs, Cherian, and Candès (Gibbs, Cherian, and Candès 2023) extend split conformal prediction beyond marginal coverage by introducing a finite-sample *conditional* calibration procedure. Vishwakarma et al. (Vishwakarma et al. 2025) apply marginal split conformal prediction directly to LLM decision-making, pruning candidate actions to a calibrated set using a single global nonconformity threshold. Our state-conditional calibration procedure builds directly on the conditional calibration machinery of Gibbs, Cherian, and Candès (2023), instantiating it over local k -nearest-neighbor groups in the agent’s state-embedding space; this addresses the state-dependent difficulty that a global threshold, as used in Vishwakarma et al. (2025), does not account for, at the cost of the exchangeability caveats discussed in Section 3.3.

3 Conformal Action Sets-based LLM Agents

We describe our framework, **Conf-ReAct**, for calibrated action selection in LLM-driven agents. Section 3.1 formalizes the planning problem. Section 3.2 describes how our multi-action, conformally-pruned exploration strategy departs from

single-action ReAct-style planning. Section 3.3 defines the two components of our online conformal post-processor – a learned score function and a state-conditional calibration procedure – and describes how each is trained and deployed at inference time.

3.1 Problem Formulation

We consider an agent operating in an interactive environment defined by a tuple $(\mathcal{D}, \mathcal{S}, \mathcal{A}, \text{Env})$:

- **Task description** $\mathcal{D}$: a natural-language specification of the task and the environment the agent operates in (e.g. an instruction to purchase a specific product in *WebShop*, a target value in *Game of 24*, or a scientific objective in *ScienceWorld*). $\mathcal{D}$ is fixed for the duration of an episode and is prepended to every reasoning prompt issued to the LLM.
- **State space** $\mathcal{S}$: at time t , the agent maintains a textual state estimate $s_t \in \mathcal{S}$ (e.g. the path to the current webpage in *webshop*, the current numbers remaining in *Game of 24*, or the path to the current room in *ScienceWorld*). The episode begins at a designated initial state s_0 which also includes the task description. Subsequent states are constructed by appending the action description to the current state.
- **Action space** $\mathcal{A}(s)$: for each state s , a (possibly state-dependent, environment-defined) finite set of textual actions available to the agent.¹
- **Environment** Env : given an internal (hidden) state s_t and an executed action a_t , the environment returns an observation and a scalar reward, $(o_{t+1}, r_{t+1}) = \text{Env}(s_t, a_t)$, and the agent’s internal state is updated, $s_t \rightarrow s_{t+1}$, via $\phi(s_t, a_t, o_{t+1})$ for some (environment-specific) state-update function ϕ . Note that, the agent does not have access to the environment’s internal state, but maintains its own textual state representation.

An episode terminates either when the environment signals task completion (success) or when a fixed step or LLM-call budget is exhausted (failure). Our goal is to design a policy that selects actions so as to maximize the probability of reaching a success state while minimizing the number of LLM calls required per episode.

3.2 Agentic Solution Architecture

ReAct-style agents (Yao et al. 2023b) interleave reasoning and acting: at each state s_t , the agent queries the LLM once to produce a single candidate action a_t , executes it, and observes (o_{t+1}, r_{t+1}) . Because the LLM’s proposed action is not verified before execution, an incorrect action can only be discovered *after* it has already been taken – committing the trajectory to a suboptimal branch, or in irreversible situations to failure. Reflection-based extensions (Reflexion, ReAct) address this by issuing *additional* LLM calls after a failed attempt to critique and revise the agent’s plan, which

¹We do not assume $\mathcal{A}(s)$ is enumerable by the environment in general; rather, the LLM proposes a finite candidate subset $\hat{\mathcal{A}}(s) \subseteq \mathcal{A}(s)$ at each step, as described in Section 3.2.

improves success rates but multiplies the number of LLM calls per episode, since each correction round is itself a full reasoning-and-acting cycle. They also require the reflection LLM agent to be a thinking model that can analyze the entire trajectory, thus increasing the prompt size and token length.

In contrast, we query the LLM *once* per state to obtain a *shortlist* of m candidate actions, $\hat{\mathcal{A}}(s_t) = \{a_t^{(1)}, \dots, a_t^{(m)}\}$. Rather than executing all m candidates uniformly, we use an online conformal post-processor (Section 3.3) to prune this shortlist to a calibrated subset $\hat{C}(s_t) \subseteq \hat{\mathcal{A}}(s_t)$ that is guaranteed, at a user-specified confidence level $1 - \alpha$, to contain the optimal next action. We then potentially explore all the selected actions in $\hat{C}(s_t)$ by storing them in a buffer and executing them in the environment and repeatedly resetting to the previous state. The key design premise is a *cost asymmetry*: LLM calls are often the dominant cost of agentic planning and often dominate wall-clock latency and inference-time expense, whereas executing an environment action is comparatively cheap. Hence executing a small, calibrated *set* of actions rather than a single uncertain guess, shifts cost away from LLM calls and onto action execution, which is the more abundant resource in most agentic deployments.

Executing $|\hat{C}(s_t)| \geq 1$ actions from s_t yields $|\hat{C}(s_t)|$ successor states, which we insert into a state buffer $\mathcal{B}$. At each round we pop a state from $\mathcal{B}$ (we use a FIFO / breadth-first order in our experiments²), issue a single LLM call to obtain its action shortlist, prune it via the conformal post-processor, execute the surviving actions, and push their successor states back onto $\mathcal{B}$. This continues until either (i) a successor state is flagged by the environment as a success state, or (ii) the LLM-call or step budget is exhausted.

Why a conformal shortlist? A naïve alternative to pruning is to simply execute all m LLM-proposed candidates at every state. This controls neither the size nor the growth rate of the state buffer $\mathcal{B}$: with branching factor m at every state, the buffer grows as $O(m^d)$ after d rounds, quickly exhausting the action budget on low-value branches. The conformal set $\hat{C}(s_t)$ instead adapts its size to the model’s confidence at s_t : when the score function is confident (the top action is clearly separated from the rest), $|\hat{C}(s_t)|$ is small and close to 1, recovering the efficiency of single-action planning; when the score function is uncertain, $|\hat{C}(s_t)|$ grows to preserve the coverage guarantee, trading additional (cheap) action executions for a higher probability of retaining the optimal action. The user-specified miscoverage level α directly controls this trade-off.

3.3 Online Conformal Post-Processor

The post-processor consists of two components, described in turn: (1) a learned *score function* $g_\theta(s, a)$ that estimates how likely an action a is to be optimal at state s , and (2) a *state-conditional calibration procedure* that converts scores into a calibrated action set $\hat{C}(s)$ with a state-conditional coverage guarantee.

²Alternative pop orders – e.g. prioritizing states by score-function confidence – are a natural extension we leave to future work.

Score Function: Let $g_\theta : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ be a two-layer feedforward network,

$$g_\theta(s, a) = \sigma(W_2 \text{ReLU}(W_1[\psi(s); \psi(a)] + b_1) + b_2),$$

where $\psi(\cdot)$ denotes a frozen text embedding of the state or action string (we use Sentence-Bert) and σ is the sigmoid function. The output $g_\theta(s, a) \in [0, 1]$ is trained to approximate the probability that a is an optimal action at s , i.e. that a lies on a successful completion trajectory from s .

Offline dataset construction: We construct the labeled training dataset $\mathcal{D}_{\text{train}} = \{(s_i, a_i, y_i)\}_{i=1}^N$ and the cache for Calibration set construction $\mathcal{D}_{\text{cache}}$ entirely offline, using tasks split into training and test. For each environment we partition its pool of task descriptions $\mathcal{D}$ (e.g. shopping instructions in *WebShop*, target tuples in *Game of 24*, goal specifications in *ScienceWorld*) into a *training task split* $\mathcal{D}_{\text{tasks}}^{\text{train}}$ and a *test task split* $\mathcal{D}_{\text{tasks}}^{\text{test}}$, with no overlap between the two. All results reported in Section 4 (success rate, LLM-call counts, coverage) are computed exclusively on $\mathcal{D}_{\text{tasks}}^{\text{test}}$; the training split is used only to build $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{cache}}$ below, and is never seen at evaluation time.

For each task in $\mathcal{D}_{\text{tasks}}^{\text{train}}$, we explore the environment starting from the task’s initial state and generate trajectories using an *auxiliary LLM* that is separate from the agent LLM used at test time. At each visited state s , the auxiliary LLM proposes a shortlist of candidate actions, exactly as the agent LLM does at inference time (Section 3.2). Each proposed action is executed to expand the trajectory, and the environment’s terminal signal (success or failure) is recorded for the resulting trajectory. We then back-label every (s, a) pair visited along these training trajectories: an action a taken at state s is labeled $y = 1$ if it lies on a trajectory that reached success, and $y = 0$ if it lies on a trajectory that terminated in failure (including actions that were proposed by the auxiliary LLM at s but not chosen, when their sub-trajectories were also rolled out and failed). This yields a binary classification dataset with positive and negative examples at each cached state. The full set of visited (s, a, y) triples across all training tasks forms $\mathcal{D}_{\text{cache}}$; a subset of these, used only for fitting g_θ , forms $\mathcal{D}_{\text{train}} \subset \mathcal{D}_{\text{cache}}$, with the remainder of $\mathcal{D}_{\text{cache}}$ held out for the state-conditional calibration procedure. This construction is performed once per environment and reused across all evaluated agent models and all choices of α .

Training objective: Given $\mathcal{D}_{\text{train}}$, we minimize binary cross-entropy: $\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log g_\theta(s_i, a_i) + (1 - y_i) \log (1 - g_\theta(s_i, a_i))]$. At inference time, given a state s_t and LLM-proposed shortlist $\hat{\mathcal{A}}(s_t)$, we rank candidates by $g_\theta(s_t, \cdot)$ and define the nonconformity score of a candidate action as

$$V(s_t, a) := 1 - g_\theta(s_t, a), \quad a \in \hat{\mathcal{A}}(s_t), \quad (1)$$

so that actions the score function judges more likely to be optimal receive lower nonconformity, consistent with standard conformal prediction convention.

State-Conditional Calibration: A single global threshold on $V(s, a)$ as in marginal split conformal prediction used in (Vishwakarma et al. 2025) guarantees only $\mathbb{P}(a^* \in \hat{C}(s)) \geq 1 - \alpha$ averaged over the distribution of states the calibration set was drawn from. This is insufficient in our setting for two reasons. First, the difficulty of correctly ranking actions is highly state-dependent (e.g. states with many plausible near-duplicate actions are intrinsically harder than states with one clearly correct action), so a marginal threshold will systematically under-cover hard states and over-cover easy ones. Second, the state distribution actually visited at test time is induced by the exploration policy of an existing LLM, since which states the agent reaches depends on which actions the conformal post-processor itself selects, the exchangeability assumption underlying standard split conformal prediction between a fixed calibration set and the test point is not immediately justified. We do not resolve this covariate-shift concern in full generality; instead, we adopt a *localized, state-conditional* calibration scheme that empirically compensates for state-dependent difficulty and provides an interpretable notion of conditional coverage.

Calibration set selection: Let $\mathcal{D}_{\text{cache}} = \{(s_j, a_j, y_j)\}_{j=1}^n$ denote the full BFS-enumerated state-action cache built from $\mathcal{D}_{\text{tasks}}^{\text{train}}$ (Section 3.3), held out from score-function training. At runtime, given the current state s_t , we select a state-localized calibration subset $\mathcal{D}_{\text{cal}}(s_t) := \{(s_j, a_j, y_j) \in \mathcal{D}_{\text{cache}} : \|\psi(s_j) - \psi(s_t)\|_2 \leq \rho_k(s_t)\}$, where $\rho_k(s_t)$ is the distance to the k -th nearest cached state to s_t under the embedding ψ ; i.e. we take the k nearest cached states to s_t (and all of their associated action examples) as the calibration set for this round.

Threshold computation via conditional calibration: Given $\mathcal{D}_{\text{cal}}(s_t)$, rather than simply taking the empirical $(1 - \alpha)$ -quantile of nonconformity scores in this local neighborhood (which would itself only be a heuristic, uncalibrated adjustment), we apply the finite-sample conditional calibration procedure of Gibbs, Cherian, and Candès (2023). Concretely, we instantiate their augmented quantile regression (their Eq. 2.4) over the finite-dimensional function class $\mathcal{F} = \{\mathbf{1}\{s \in G_1\}, \dots, \mathbf{1}\{s \in G_k\}\}$, where $G_1, \dots, G_k$ are the k groups of cached states selected as each cached state’s k -nearest-neighborhood under ψ ;³ that is, we solve

$$\begin{aligned} (\hat{\eta}^S)^* &= \arg \max_{\eta \in \mathbb{R}^{n+1}} \sum_{i=1}^n \eta_i V_i + \eta_{n+1} S - \mathcal{R}^*(\eta) \\ \text{s.t.} \quad & -\alpha \leq \eta_i \leq 1 - \alpha \end{aligned} \quad (2)$$

following the dual formulation of Gibbs, Cherian, and Candès (2023, Eq. 4.2), restricted to the localized calibration set $\mathcal{D}_{\text{cal}}(s_t)$ in place of the full calibration set. Using the monotonicity of $S \mapsto \hat{\eta}_{n+1}^S$ established there (their Theorem 4),

³special case (their Corollary 1) rather than the general RKHS/Lipschitz extension in their Sections 3.1–3.2; a kernel-weighted variant (their Section 3.1, using a Gaussian kernel over state embeddings in place of hard k -NN groups) is a natural refinement if hard neighborhoods prove too coarse in practice.

Table 1: Performance of planning methods on different environments and LLMs. **Conf-ReAct** is evaluated at 90%, 80%, and 70% target coverage. LLM calls denotes avg. number of calls made per winning episode.

Model	Method	WebShop			Game of 24		ScienceWorld		
		Avg score	Success Rate	LLM Calls	Success Rate	LLM Calls	Avg Score	Success Rate	LLM Calls
Qwen-2.5-3B	ReAct (Yao et al. 2023b)	13.3	10%	7.0	4%	8.0	15.1	4.4%	10.1
	Reflexion (Shinn et al. 2023)	58.52	29%	21.72	7%	11.5	18.7	6.5%	20.2
	RefAct (Kim et al. 2025)	64.68	36%	6.50	3%	8.0	15.3	5%	9.8
	Rollback (Li et al. 2025)	59.39	28.5%	10.04	7%	22.00	19.7	7%	25.3
	Conf-ReAct (90%)	78.7	59%(+23%)	30.80	28%(+21%)	5.9	21.7	10%(+3%)	19.4
	Conf-ReAct (80%)	74.3	52%(+16%)	23.43	24%(+17%)	4.62	16.2	9%(+2%)	18.6
	Conf-ReAct (70%)	72.1	50.5%(+14%)	11.51	14%(+7%)	3.29	5.5	8%(+1%)	17.4
Qwen-3-8B	ReAct	51.48	25%	6.00	5%	8.46	27.2	20%	8.8
	Reflexion	55.61	29%	11.00	12%	14.12	40.1	26%	17.2
	RefAct	39.23	24%	5.92	10%	8.52	28	23%	8.6
	Rollback	56.35	36%	9.2	11%	31.00	32	27%	20.1
	Conf-ReAct (90%)	80.2	61.5%(+25%)	22.37	38%(+26%)	7.36	60.9	32%(+5%)	12.2
	Conf-ReAct (80%)	76.5	58.5%(+22%)	16.31	29%(+17%)	4.6	54.2	27%(+0%)	11.6
	Conf-ReAct (70%)	70.5	51.5%(+15%)	14.54	23%(+11%)	3.55	53.1	25%(-2%)	10.8
Gemma-4-12B	ReAct	51.4	36.4%	5.47	10%	8.4	45.2	32%	6.2
	Reflexion	57.6	40%	7.59	15%	11.82	50.1	35%	10.6
	RefAct	49.5	34%	5.43	11%	8.00	42.6	33%	7.5
	Rollback	69.8	45%	8.54	12%	15.5	55.2	37%	12.1
	Conf-ReAct (90%)	81.1	67%(+22%)	11.95	50%(+35%)	6.6	58.5	44%(+7%)	6.8
	Conf-ReAct (80%)	78.29	62%(+17%)	11.02	38%(+23%)	5.85	55.6	38%(+1%)	6.0
	Conf-ReAct (70%)	72.1	57%(+12%)	8.41	29%(+14%)	5.03	51.1	33%(-4%)	5.0

we recover the group-conditional threshold $\tau_\alpha(s_t)$ for the neighborhood containing s_t via their two-step procedure (binary search or LP sensitivity analysis over S), and define the conformal action set

$$\hat{C}(s_t) := \{a \in \hat{\mathcal{A}}(s_t) : V(s_t, a) \leq \tau_\alpha(s_t)\}. \quad (3)$$

Restricting calibration to the local neighborhood $\mathcal{D}_{\text{cal}}(s_t)$ at each round means the resulting threshold $\tau_\alpha(s_t)$ adapts to the difficulty of the current state’s neighborhood, rather than reflecting a single global average difficulty across the whole cache.

Limitations of this guarantee: We emphasize that the coverage guarantee inherited from Gibbs, Cherian, and Candès (2023) is with respect to a test point exchangeable with the (localized) calibration set $\mathcal{D}_{\text{cal}}(s_t)$, under the assumption that this calibration set and the current state s_t are drawn from a common distribution. In our setting, $\mathcal{D}_{\text{cal}}(s_t)$ is drawn from an offline, auxiliary-LLM-generated cache over the training task split, while s_t is a state reached online, at test time, by our own exploration policy over held-out test tasks; as discussed above, these need not be exchangeable in general, since the policy’s own pruning decisions influence which states are subsequently visited. We report empirical coverage against this target as an approximate, rather than formally guaranteed, quantity in this setting, and view establishing a rigorous non-exchangeable coverage bound (in the spirit of Barber et al. (2023)) for this online, policy-induced state distribution as important future work.

Role of α : The miscoverage level α is the single user-facing control knob of **Conf-ReAct**, exposing the trajectory-length / success-rate trade-off discussed in Section 3.2 directly to the practitioner. Small α (high target coverage, e.g.

$1 - \alpha = 0.9$) yields larger conformal sets $\hat{C}(s_t)$, increasing the number of actions executed per round and hence the number of states explored, which raises the probability that the optimal trajectory survives pruning at the cost of more environment interactions per LLM call. Large α (e.g. $1 - \alpha = 0.7$) yields smaller, more aggressively pruned sets, reducing exploration cost but increasing the risk of discarding the optimal action at some step along the trajectory. Because $\tau_\alpha(s_t)$ is recomputed at every state from the local calibration neighborhood, a single choice of α automatically yields state-dependent set sizes - small at easy, well-separated states and large at ambiguous ones - without requiring the user to hand-tune a schedule.

4 Experimental Results

In this section, we report experimental results validating the proposed technique and comparing it to state of the art baselines.

4.1 Experimental Setup

Environments: We evaluate our approach on three different environments: **(1) WebShop** (Yao et al. 2022): It is a simulated e-commerce environment that evaluates an agent’s ability to understand a user’s shopping intent and perform step-by-step planning in a web-navigation environment to identify and purchase the correct product. It contains approximately 1.18 million real-world products and more than 12,000 crowd-sourced shopping instructions. **(2) Game of 24** (Yao et al. 2023a): It is a mathematical reasoning benchmark in which an agent is given four integers and must generate a sequence of arithmetic operations (eg. $+$, $-$, $\times$, and $\div$) such that the result evaluates exactly to 24. **(3) ScienceWorld** (Wang et al. 2022): It is a text-based interactive en-

Table 2: Empirical coverage of **Conf-ReAct** against Reflexion on different environments and LLMs, broken down by prediction-set-size bin (number of actions kept). **Conf-ReAct** rows report target coverage $1 - \alpha \in \{90\%, 80\%, 70\%\}$; Reflexion rows T1–T3 are successive trials, with T1 corresponding to plain ReAct. *N/A* indicates bins with no states.

Model	Method	Game of 24			WebShop			ScienceWorld		
		(0–2]	(2–4]	(4–16]	(0–2]	(2–4]	(4–16]	(0–2]	(2–4]	(4–16]
Qwen2.5-3B	ReAct/Reflexion T1	33%	58%	N/A	15%	20%	N/A	55%	37%	N/A
	Reflexion T2	29%	58%	N/A	16%	28%	N/A	30%	40%	N/A
	Reflexion T3	30%	58%	N/A	18%	24%	N/A	31%	37%	N/A
	Conf-ReAct (90%)	67%	72%	94%	82%	100%	100%	95%	97%	100%
	Conf-ReAct (80%)	51%	79%	85%	74%	82%	81%	88%	87%	95%
	Conf-ReAct (70%)	41%	67%	74%	71%	72%	75%	79%	82%	88%
Qwen3-8B	ReAct/Reflexion T1	50%	52%	N/A	50%	50%	N/A	95%	67%	N/A
	Reflexion T2	40%	42%	N/A	29%	35%	N/A	85%	69%	N/A
	Reflexion T3	57%	71%	N/A	38%	31%	N/A	86%	68%	N/A
	Conf-ReAct (90%)	55%	71%	100%	93%	93%	92%	98%	100%	100%
	Conf-ReAct (80%)	43%	64%	87%	85%	80%	81%	87%	90%	100%
	Conf-ReAct (70%)	39%	55%	64%	74%	71%	68%	81%	95%	97%
Gemma-4-12B	ReAct/Reflexion T1	41%	79%	N/A	41%	N/A	N/A	65%	N/A	N/A
	Reflexion T2	46%	70%	N/A	38%	N/A	N/A	67%	72%	N/A
	Reflexion T3	48%	53%	N/A	38%	N/A	N/A	67%	59%	N/A
	Conf-ReAct (90%)	55%	62%	60%	97%	100%	100%	97%	100%	100%
	Conf-ReAct (80%)	55%	51%	69%	94%	100%	100%	89%	95%	100%
	Conf-ReAct (70%)	45%	48%	65%	89%	100%	N/A	80%	90%	100%

environment containing diverse science-oriented tasks. It evaluates an agent’s ability to reason about scientific concepts, perform long-horizon planning, and interact with a dynamic environment to accomplish specified goals.

Constructing the Prediction Set: The threshold function receives three inputs: the current state’s embedding, the pool of past calibration states each paired with the non-conformity score of the action taken there, and the score that our score function assigns to a candidate action. It retains the n calibration states closest to the current one, fits a local linear program over their scores, and asks whether the candidate’s score sits at or below the conditional quantile implied by those neighbours. Repeating this once per admissible action returns the prediction set: the subset of actions the candidate score admits at confidence $1 - \alpha$. Because the neighbourhood is re-selected at every state, the effective threshold adapts to local difficulty rather than being a single fixed cutoff.

4.2 Baseline Comparison w.r.t. task-completion

Table 1 represents performance of planning methods on WebShop, Game of 24, and ScienceWorld. The column *Success Rate* is the task-success rate; *LLM Calls* is the average number of LLM calls per winning episode. It shows that **Conf-ReAct** raises the success rate over every baseline across all three environments and all three backbones. At the 90% coverage level, **Conf-ReAct** achieves improvements of **25%** on WebShop, **35%** on Game of 24, and **7%** on ScienceWorld relative to the strongest baseline. The improvement holds as the LLM grows from 3B to 12B parameters, indicating that the benefit of filtering actions through a calibrated uncertainty estimate is remains relevant even for some of

the strongest and latest small language models. These gains, however, come at the price of additional inference. Because a higher target coverage $1 - \alpha$ retains a larger set of candidate actions for the planner to explore, **Conf-ReAct** issues more LLM calls per successful episode as coverage tightens on WebShop with Qwen-2.5-3B, for instance, the budget rises from 11.51 calls at 70% coverage to 30.80 at 90%. This exposes a controllable accuracy-cost tradeoff governed by a single interpretable knob, the miscoverage level α : driving α down toward 0.1 (i.e., 90% coverage) favors completion at the expense of more queries, while raising it toward 0.3 (70% coverage) recovers most of the baseline efficiency at a small penalty in success.

4.3 Validation of conditional-conformal sets

We ask the question *-does **Conf-ReAct** produce action sets with higher coverage for same number of selected actions?* Table 2 reports the empirical coverage of (conformal) action-sets produced by **Conf-ReAct** and the baseline methods, bucketed into different size-buckets. Across WebShop and ScienceWorld and every LLM-backbone, **Conf-ReAct** at the 90% target ($\alpha = 0.1$) improves the coverage over the baselines, and does so most sharply in the smallest (0–2] bin. In many cases, the overall improvement in coverage exceeds 40% showing the strong performance. Notably, repeated reflexion actually decreases the coverage, likely because SLMs are not very good at filtering actions from long contexts. Notwithstanding the concern about distribution drift, we see that target coverage guarantee is mostly met by the empirical coverage from **Conf-ReAct** across buckets on Webshop and ScienceWorld datasets, bolstering our claim that the pro-

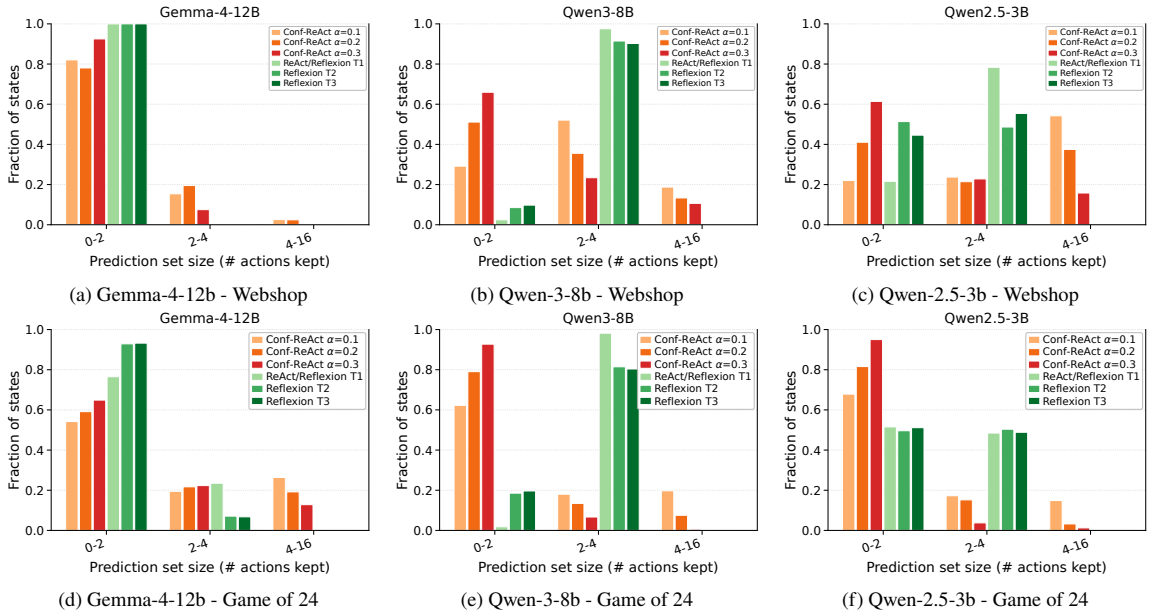


Figure 2: State distribution as a function of the size of the LLM-generated action set, for baselines and various α for **Webshop** (2a, 2b, 2c), and **Game of 24** (2d, 2e, 2f) for Gemma-4-12b, Qwen-3-8b and Qwen-2.5-3b.

posed pipeline of training scoring function, and constructing calibration sets are useful. We attribute the lower empirical coverage on Game of 24 to the character of the task, which demands exact arithmetic composition, and evaluation. This may not be effectively processed

Figure 2 plots the distribution of states over prediction-set-size bins: across all three environments and backbones. **Conf-ReAct** places the majority of its states in the (0–2] bucket, while the Reflexion baselines have relatively larger fraction of states in (2–4] bins. However, for lower values of α , **Conf-ReAct** generates states with large number of actions, which also explains the overall increase LLM calls. Overall, we can conclude that with **Conf-ReAct** a smaller retained set does not come *at the cost* of coverage; and the increase in action set size is commensurate with the conformal guarantee.

Effect of α : In table 2, the empirical coverage of the all the action sets grow as α is reduced. For instance, Qwen-3-8B on WebShop has coverage it ranging from **74%** at $\alpha = 0.3$ to **85%** at $\alpha = 0.2$ and **93%** at $\alpha = 0.1$, and the same trend holds for every model and environment. Table 1 tells the matching story for success rate: on Game of 24 with Gemma-4-12B it rises from **29%** to **38%** to **50%** as α falls from 0.3 to 0.1. Figure 2 shows that as α shrinks, the state mass shifts out of the smallest (0–2] bin into the larger (2–4] and (4–16] bins. For Qwen-3-8B on WebShop the fraction of states in the smallest bin drops from roughly **0.65** at $\alpha = 0.3$ to about **0.29** at $\alpha = 0.1$. Hence, α can be used to predictably tradeoff between completion rate and cost in terms of LLM calls. The effect of α , depends on the task, with webshop shifting the mass more since there are many more potential correct actions (queries), and opposed to Game of 24, where the correct actions are mostly limited and well-defined.

5 Conclusion

We introduced **Conf-ReAct**, a conformal-prediction framework that equips ReAct-style LLM agents with a statistically calibrated set of candidate actions at each state rather than a single uncertain guess. A learned nonconformity score function ranks actions by their likelihood of lying on a successful trajectory, and a state-conditional calibration procedure adapts the pruning threshold to local state difficulty, with a breadth-first planner exploring the retained set. Across WebShop, Game of 24, and ScienceWorld and three backbones from 3B to 12B parameters, **Conf-ReAct** improves task-success rate over ReAct, Reflexion, ReflAct, and Rollback through calibrated ranking rather than state inflation, while exposing the accuracy–cost trade-off through a single interpretable knob α . Limitations include the exchangeability caveat for our online, policy-induced state distribution and degraded scoring on symbolic tasks like Game of 24, both of which we leave, alongside confidence-prioritized exploration, to future work.

References

- Barber, R. F.; Candès, E. J.; Ramdas, A.; and Tibshirani, R. J. 2023. Conformal Prediction Beyond Exchangeability. *arXiv preprint arXiv:2202.13415*.
- Erdogan, L. E.; Lee, N.; Kim, S.; Moon, S.; Furuta, H.; Anumanchipalli, G.; Keutzer, K.; and Gholami, A. 2025. Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.
- Gibbs, I.; Cherian, J. J.; and Candès, E. J. 2023. Conformal Prediction with Conditional Guarantees. *arXiv preprint arXiv:2305.12616*.

Kim, J.; Rhee, S.; Kim, M.; Kim, D.; Lee, S.; Sung, Y.; and Jung, K. 2025. Reflect: World-grounded decision making in llm agents via goal-state reflection. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 33421–33453.

Li, P.; and Others. 2025. Rollback: Error Correction for LLM Agents via State Reversion. *arXiv preprint arXiv:2501.00000*.

Li, X.; Chen, K.; Long, Y.; Bai, X.; Xu, Y.; and Zhang, M. 2025. Generator-Assistant Stepwise Rollback Framework for Large Language Model Agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 17683–17700. Suzhou, China: Association for Computational Linguistics. ISBN 979-8-89176-332-6.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36: 8634–8652.

Vishwakarma, H.; Mishler, A.; Cook, T.; Dalmasso, N.; Raman, N.; and Ganesh, S. 2025. Prune ’n Predict: Optimizing LLM Decision-making with Conformal Prediction. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*. PMLR.

Wang, R.; Jansen, P.; Côté, M.-A.; and Ammanabrolu, P. 2022. ScienceWorld: Is your Agent Smarter than a 5th Grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 11279–11298. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, volume 35, 24824–24837. Curran Associates, Inc.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35: 20744–20757.

Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. 2023a. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *arXiv:2305.10601*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023b. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Zhao, Q.; Fu, H.; Sun, C.; and Konidaris, G. 2024. EPO: Hierarchical LLM Agents with Environment Preference Optimization. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 6401–6415. Miami, Florida, USA: Association for Computational Linguistics.